

SMART RESOURCE ALLOCATION AND OPTIMIZATION SYSTEM

GROUP ASSIGNMENT REPORT

A Group Assignment Report submitted in partial fulfilment of the requirements

for the Course of

CSA0602- Design and Analysis of Algorithms

Particular	Details
Name	J. Renuka Devi
Register number	192573007
Department:	CSE DS
Programme:	CSE
Course Code & Course Name	CSA0602 - Design and Analysis of Algorithms
Academic Year / Batch	2026
Faculty Name	Dr.P.Sriramya
Assignment Title	Smart Resource Allocation and Optimization System
Date of Issue	31/08/2026
Date of Submission	02/09/26
Maximum Marks	100
Course Outcome(s) – CO	CO4 - Solve optimization problems using dynamic programming and greedy strategies.
Bloom's Taxonomy Level	L3 – Apply, L5 – Evaluate
SDG Mapping (SDG 1-17)	SDG8 - Decent Work and Economic Growth & SDG9 - Industry, Innovation and Infrastructure
Industry / Societal Relevance	

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Jaiwanthy V (192565034)	0/1 Knapsack & Project Optimization	Designed and implemented DP algorithm, DP table, backtracking, project selection and resource optimization	Tested different resource capacities, analysed time and space complexity, validated optimization results	Problem Formulation, DP Algorithm, DP Pseudocode, DP Table, Complexity Analysis, Optimization Results	
Renuka Devi J (192573007)	Greedy Strategy & Job Sequencing	Implemented Greedy Job Sequencing and Greedy ratio-based Knapsack heuristic	Developed DP vs. Greedy comparison, counterexample, dataset testing, execution-time and scalability analysis	Greedy Algorithm, Job Sequencing Pseudocode, DP vs. Greedy Comparison, Counterexample, Performance Testing, Trade-offs	
Preethi R (192511352)	Web Application & System Integration	Developed Flask web application, UI/UX, algorithm integration,	Performed UI, integration and application testing; verified optimization	Implementation, Environment & Tools, System Architecture, UI/UX, Novel Features, SDG	

		visualizations and decision- support features	and comparison outputs	Relevance, Screenshots, Integration Testing	
Total					100%

A. Assignment Problem / Challenge

Smart Resource Allocation and Optimization System Using Dynamic Programming and Greedy Strategies

1. Problem Statement

Organizations frequently have to make decisions about how to allocate **limited resources** among multiple projects and schedule tasks within deadlines while maximizing overall benefit or profit. When the number of projects and tasks increases, making these decisions manually becomes difficult and may result in inefficient resource utilization, missed deadlines, or reduced profit.

Consider an organization that has a limited amount of resources such as **budget, manpower, machine hours, or processing capacity**. It has several projects, where each project requires a certain amount of resources and generates a corresponding expected profit. The organization must select a suitable combination of projects such that the **total resource consumption does not exceed the available capacity while maximizing the total profit**.

At the same time, the organization has multiple jobs or tasks that need to be completed within specified deadlines. Each job has a deadline and an associated profit. Since only a limited number of jobs can be completed within the available time slots, the organization needs to determine the **optimal sequence of jobs that maximizes the total profit while satisfying all deadlines**.

The proposed **Smart Resource Allocation and Optimization System** should provide algorithmic solutions to these two optimization problems using:

1. Dynamic Programming – 0/1 Knapsack

1. Select the optimal combination of projects.
2. Maximize total profit within the available resource capacity.
3. Construct and display the DP table.
4. Identify the projects selected in the optimal solution.

2. Greedy Strategy – Job Sequencing with Deadlines

1. Select jobs based on their deadlines and profits.
2. Schedule jobs in appropriate time slots.
3. Maximize the total profit while ensuring that jobs are completed before their deadlines.
4. Display the selected job sequence and total profit.

3. Input Requirements

The system should accept:

Project information:

- Project ID
- Project name
- Resource requirement
- Expected profit
- Total available resource capacity

Job information:

- Job ID
- Job name
- Deadline
- Profit
- Available time slots

4. Expected Output

The system should display:

- Optimal projects selected using **Dynamic Programming**

- Maximum achievable project profit
- Complete **DP table**
- Jobs selected using the **Greedy strategy**
- Optimal job sequence
- Maximum job profit
- Total resource utilization
- Unused resource capacity
- Algorithm execution time
- Time and space complexity

5. Performance Analysis

The system should be tested using **small, medium, and large datasets**. Students should compare the performance of Dynamic Programming and Greedy approaches in terms of:

- Execution time
- Space requirements
- Solution quality
- Scalability
- Optimality
- Resource utilization

Students should also provide **counterexamples where appropriate** to demonstrate that a greedy strategy does not always guarantee an optimal solution for problems such as 0/1 Knapsack.

6. Key Challenge

The major challenge is to determine **which optimization strategy is appropriate for a given problem and justify the choice based on the characteristics of the problem, computational complexity, and quality of the resulting solution.**

7. Expected Outcome

By completing this project, students will be able to apply Dynamic Programming and Greedy strategies to real-world optimization problems, construct and interpret algorithmic solutions, evaluate their performance, and justify the suitability of an optimization technique for a given scenario.

1. PROBLEM STATEMENT AND PROBLEM FORMULATION

1.1 Problem Statement

In real-world organizations, resources such as budget, workforce, computing capacity, equipment, and time are limited, while the number of possible projects and activities is often large. Selecting the right combination of projects is therefore an optimization problem.

A simple selection based only on individual project profitability may not produce the best overall result because a combination of projects can provide greater total profit while satisfying the available resource constraint.

Similarly, when multiple jobs have deadlines and different profits, they must be scheduled carefully so that high-value jobs are completed before their deadlines.

The OptiVerse AI – Smart Resource Allocation and Optimization System addresses these problems through two classical algorithmic approaches:

- 0/1 Knapsack using Dynamic Programming for optimal project selection.
- Job Sequencing with Deadlines using Greedy strategy for profit-maximizing job scheduling.

The system extends these algorithms into an interactive web application that provides optimization results, DP-table visualization, algorithm comparison, what-if analysis, priority intelligence, performance evaluation, and decision-support insights.

1.2 Problem Formulation

Project Optimization

Given:

- n projects
- Resource requirement r_i for each project
- Expected profit p_i for each project
- Total available resource capacity C

The objective is:

$$\text{Maximize } \sum_{i=1}^n p_i x_i$$

subject to:

$$\sum_{i=1}^n r_i x_i \leq C$$

where:

$$x_i \in \{0, 1\}$$

Here, $x_i = 1$ means the project is selected and $x_i = 0$ means it is not selected.

This represents the 0/1 Knapsack Problem because each project can either be selected completely or rejected.

Job Scheduling

For job sequencing, each job has:

- Job ID
- Job name
- Deadline
- Profit

The objective is to select and arrange jobs into available time slots so that:

$$\text{Total Profit} = \sum \text{Profit of Scheduled Jobs}$$

is maximized while every selected job is scheduled before or on its deadline.

2. OBJECTIVE AND EXPECTED OUTCOMES

2.1 Objectives

The major objectives of the project are:

1. To implement the **0/1 Knapsack algorithm using Dynamic Programming**.
2. To determine the optimal combination of projects under a fixed resource capacity.
3. To generate and visualize the complete DP table used during optimization.
4. To identify the selected projects and calculate maximum achievable profit.
5. To implement **Job Sequencing with Deadlines using a Greedy algorithm**.
6. To maximize job profit while satisfying deadline constraints.
7. To compare Dynamic Programming and Greedy approaches in terms of execution time, resource utilization, and solution quality.
8. To demonstrate a counterexample where Greedy fails to obtain the optimal Knapsack solution.
9. To provide interactive what-if analysis for different resource capacities.
10. To provide additional decision-support features through a web-based interface.

2.2 Expected Outcomes

The system is expected to:

- Produce an optimal project selection.
- Maximize project profit under resource constraints.
- Generate a complete DP matrix.
- Produce a feasible job sequence.
- Demonstrate the difference between optimal and heuristic approaches.
- Measure algorithm execution time.
- Display resource utilization and unused capacity.
- Provide visual and interactive decision support.
- Improve understanding of algorithmic trade-offs through experimentation.

3. REQUIREMENTS, CONSTRAINTS AND ASSUMPTIONS

3.1 Functional Requirements

The system should allow users to:

- Add, edit and remove projects.
- Add, edit and remove jobs.
- Specify available resource capacity.
- Execute the Dynamic Programming optimizer.
- View selected projects.
- View maximum profit.
- View resource utilization.
- View the complete DP table.
- Execute job scheduling.
- View the optimal job sequence.
- Compare DP and Greedy results.
- Run a Greedy failure counterexample.
- Perform what-if simulations.
- View performance results.
- View project priority scores.
- Export project/job information.
- View SDG-related impact information.

3.2 Non-Functional Requirements

- User-friendly interface.
- Responsive web design.
- Fast algorithm execution.
- Clear visualization of results.
- Reliable handling of user inputs.
- Modular source code.
- Easy maintainability and extension.

3.3 Constraints

The project operates under the following constraints:

- Each project is either selected or not selected.
- A project cannot be partially selected.
- Total selected resource usage cannot exceed capacity.
- Each job occupies one time slot.
- A scheduled job must satisfy its deadline.
- Greedy scheduling does not guarantee global optimality for every optimization problem.
- DP complexity depends on both the number of projects and resource capacity.

3.4 Assumptions

- Project resource requirements and profits are represented using non-negative integer values.
- Resource capacity is a non-negative integer.
- Project profit represents expected benefit.
- Job profit represents the value obtained by completing the job.
- Each job requires one time slot.
- Input data is assumed to be reasonably valid.
- Execution time can vary slightly between runs because of system conditions.

4. APPLICATION OF RELEVANT COURSE KNOWLEDGE / CONCEPTS

The project directly applies concepts from **Design and Analysis of Algorithms**.

Course Concept	Application in Project
Dynamic Programming	0/1 Knapsack optimization
Optimal Substructure	Knapsack subproblems
Overlapping Subproblems	Repeated capacity-based calculations
Greedy Strategy	Job sequencing
Sorting	Jobs sorted according to profit
Backtracking	Recovering selected projects from DP table
Time Complexity	Algorithm performance analysis
Space Complexity	DP memory analysis
Algorithm Comparison	DP vs Greedy
Experimental Analysis	Performance testing
Optimization	Maximum profit under constraints

Dynamic Programming

The Knapsack problem exhibits:

- **Optimal substructure**
- **Overlapping subproblems**

Therefore, Dynamic Programming is appropriate.

Greedy Strategy

Job Sequencing follows a greedy principle:

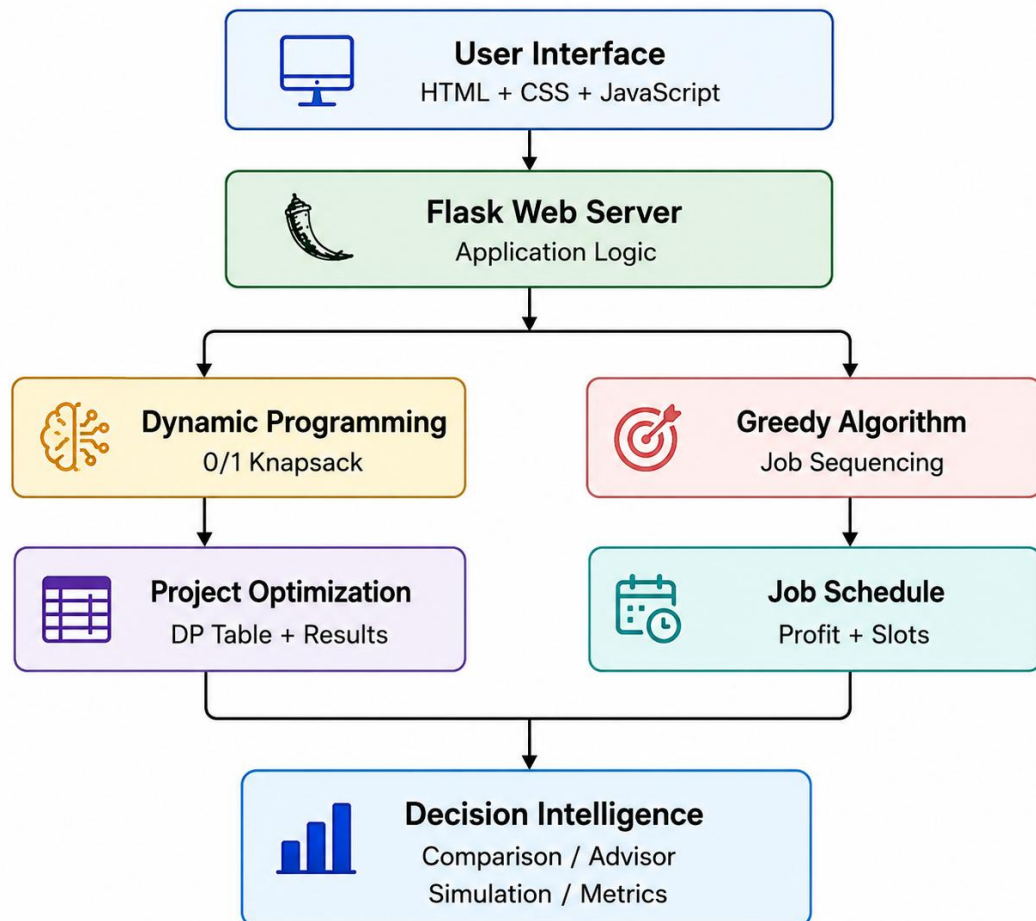
Select the highest-profit available job and place it in the latest available slot before its deadline.

This provides an efficient scheduling strategy.

5. DESIGN / PROPOSED SOLUTION / METHODOLOGY

5.1 Proposed Architecture

The system follows a layered web application structure:



6. SYSTEM MODULES

6.1 Dashboard Module

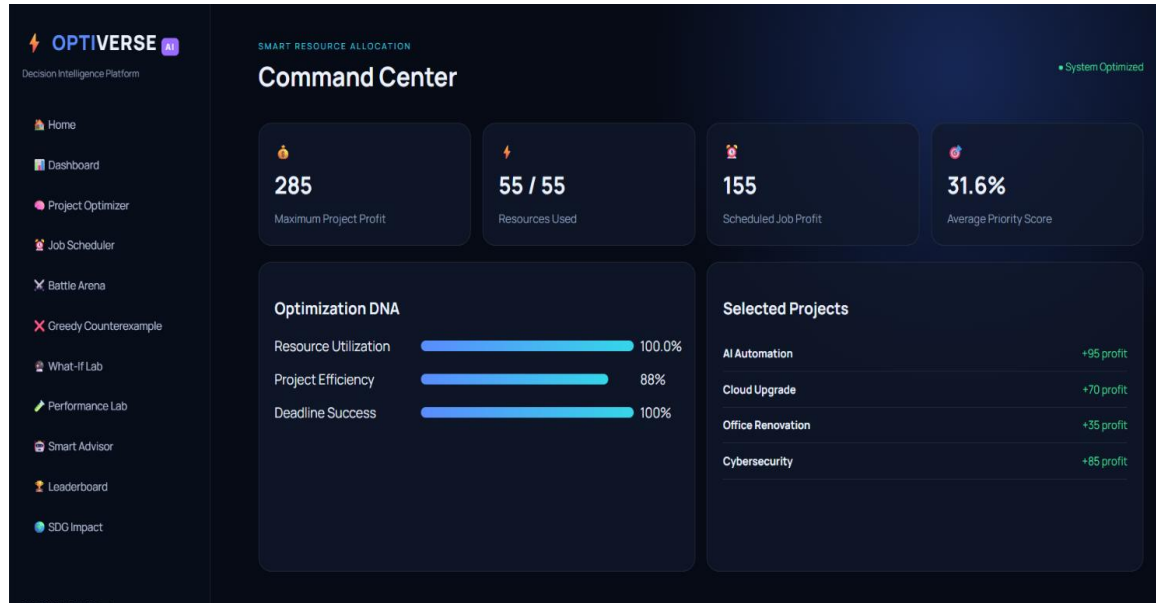
Provides a centralized overview of:

- Total projects
- Total jobs
- Available resources
- Optimization status
- Profit metrics
- Algorithm insights

Figure 1: OptiVerse AI Dashboard

6.2 Project Optimizer

The Project Optimizer is the main Dynamic Programming module.



Input:

- Project ID
- Project name
- Resource requirement
- Expected profit
- Resource capacity

Output:

- Selected projects
- Maximum profit
- Used resources
- Unused resources
- DP table
- Execution time

- Complexity

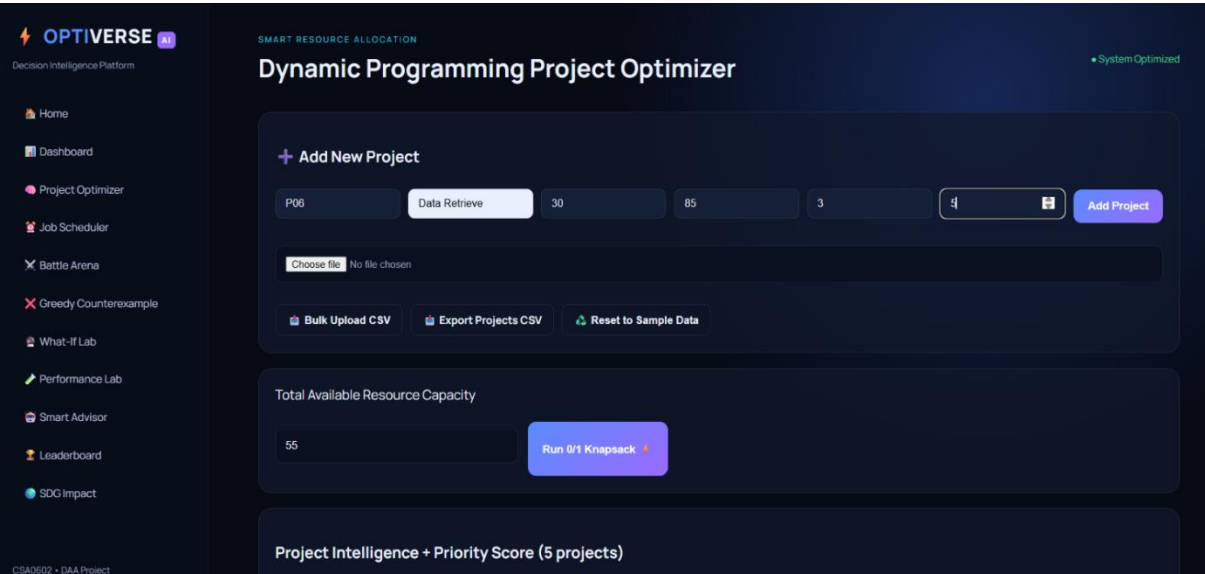
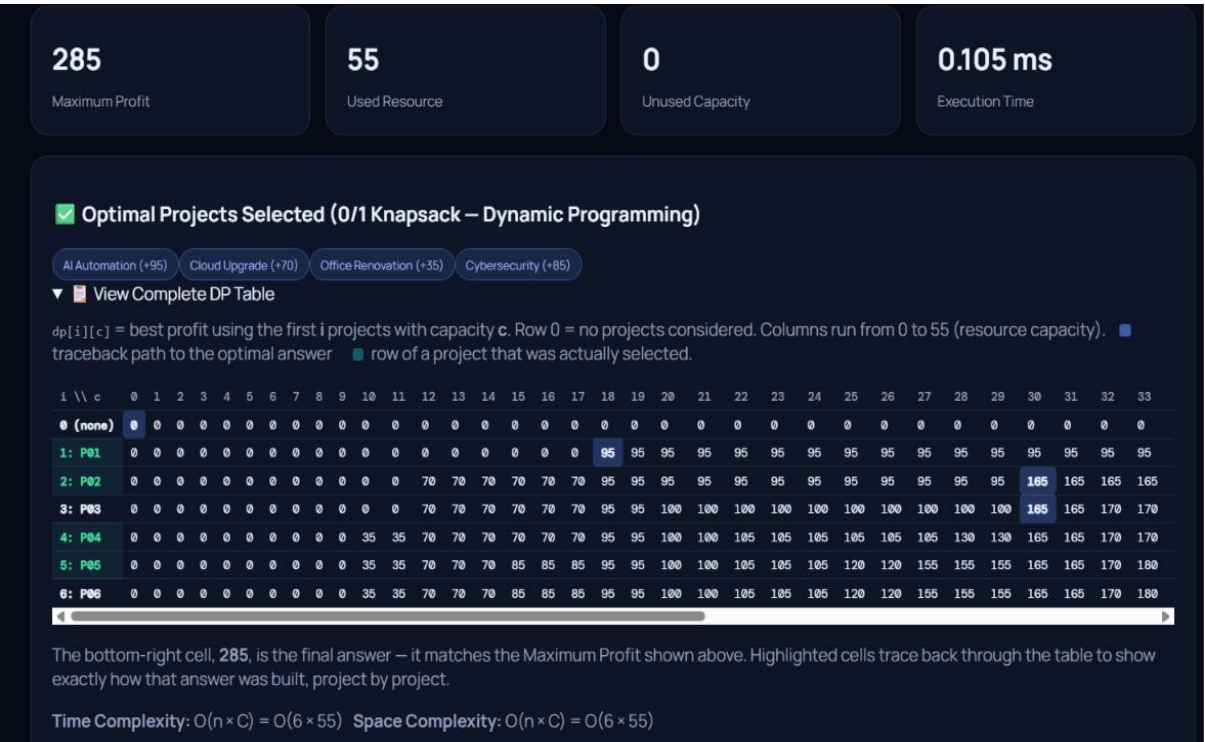


Figure 2: Project Optimizer Input and Output

6.3 Job Scheduler



Total Available Resource Capacity

55

Run 0/1 Knapsack ⚡

Project Intelligence + Priority Score (6 projects)

ID	Project	Resource	Profit	Risk	Priority	
P01	AI Automation	18	95	2/5	35.5/100	Delete
P02	Cloud Upgrade	12	70	3/5	32.5/100	Delete
P03	Green Energy	20	100	2/5	34.8/100	Delete
P04	Office Renovation	10	35	4/5	18.6/100	Delete
P05	Cybersecurity	15	85	2/5	36.6/100	Delete
P06	Data Retrieve	30	85	3/5	27.3/100	Delete

The Job Scheduler implements Job Sequencing with Deadlines using Greedy strategy.

It:

1. Sorts jobs according to profit.
2. Examines jobs in descending profit order.
3. Searches for the latest available slot before the deadline.
4. Assigns the job if a valid slot exists.
5. Calculates Total profit

Decision Intelligence Platform

- Home
- Dashboard
- Project Optimizer
- Job Scheduler
- Battle Arena
- Greedy Counterexample
- What-If Lab
- Performance Lab
- Smart Advisor
- Leaderboard
- SDG Impact

CSA0602 • DAA Project
DP • Greedy Strategies

Run Job Sequencing (Greedy)

Job List (6 jobs)

ID	Job	Deadline	Profit	
J01	Data Backup	2	40	Delete
J02	Security Patch	1	70	Delete
J03	Server Migration	2	55	Delete
J04	Report Generation	3	30	Delete
J05	System Audit	1	25	Delete
J06	Image processing	3	89	Delete

214
Maximum Job Profit

3
Jobs Scheduled

3
Time Slots Used

0.0291 ms
Execution Time

214
Maximum Job Profit

3
Jobs Scheduled

3
Time Slots Used

0.0291 ms
Execution Time

✓ Optimal Job Sequence (Greedy Strategy – Job Sequencing with Deadlines)

Security Patch (+70, due 1) Server Migration (+55, due 2) Image processing (+89, due 3)

Slot 1
Security Patch
+70 profit

Slot 2
Server Migration
+55 profit

Slot 3
Image processing
+89 profit

Time Complexity: $O(n \log n + n \times d)$ Space Complexity: $O(d)$

6.4 Algorithm Battle Arena

This module directly compares:

- Dynamic Programming
- Greedy Knapsack

Metrics include:

- Maximum profit
- Resource used
- Unused capacity
- Execution time
- Solution quality

The screenshot shows the 'Algorithm Battle Arena' interface. On the left is a sidebar with navigation links: Home, Dashboard, Project Optimizer, Job Scheduler, Battle Arena, Greedy Counterexample, What-If Lab, Performance Lab, Smart Advisor, Leaderboard, and SDG Impact. The main area is titled 'Algorithm Battle Arena' and compares two algorithms:

- Dynamic Programming** (Score: 265): Guaranteed optimal solution. Metrics: Execution: 0.0986 ms, Resource: 50/50, Optimality: ☒ Guaranteed, Memory: Higher.
- Greedy Heuristic** (Score: 250): Fast ratio-based selection. Metrics: Execution: 0.017 ms, Resource: 45/50, Optimality: ☐ Not guaranteed, Memory: Lower.

A 'Smart Verdict' box at the bottom states: 'Dynamic Programming wins – choose based on the trade-off between guaranteed optimality and computational efficiency.'

6.5 Greedy Counterexample Lab

The system includes a specifically designed example to demonstrate why a ratio-based Greedy method cannot guarantee an optimal solution for 0/1 Knapsack.

The screenshot shows the 'Greedy Failure Counterexample' interface. It features a text box stating: 'This page proves – with an actual computed example, not just theory – that the profit/resource-ratio Greedy heuristic does **not** always find the optimal solution for the 0/1 Knapsack problem, while Dynamic Programming always does.'

Below this is a table titled 'Dataset (Capacity = 50)' with the following data:

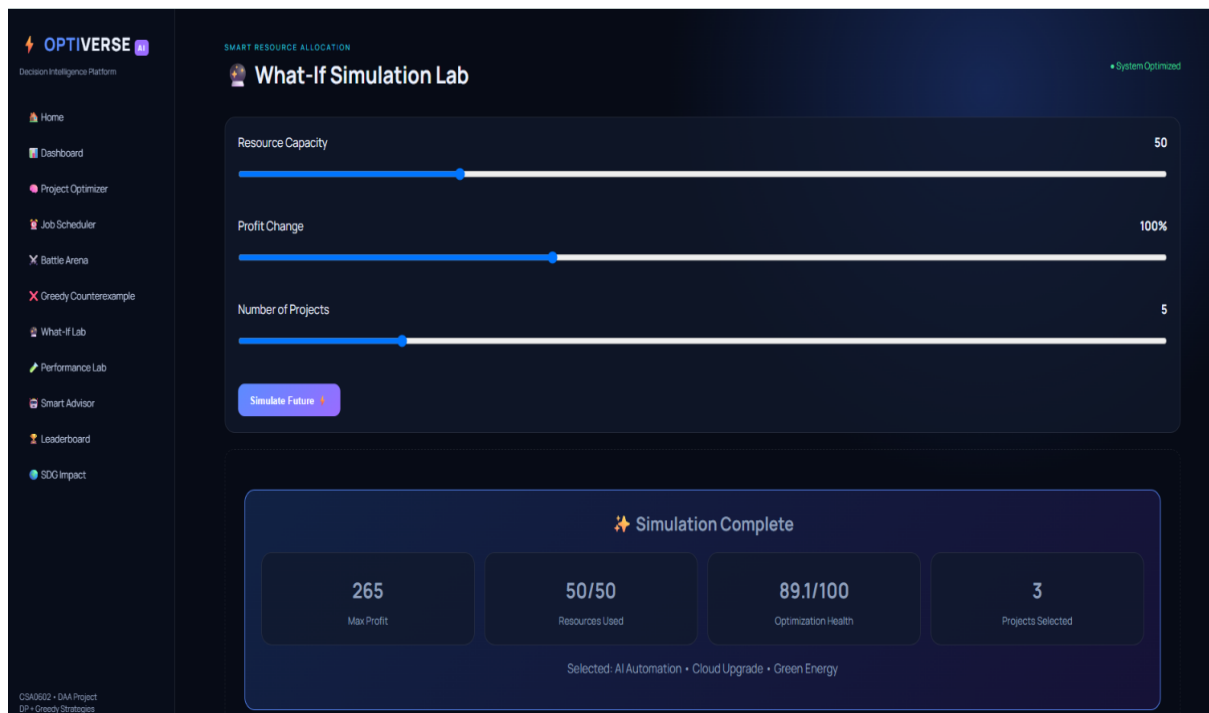
ID	Name	Resource	Profit	Profit/Resource Ratio
C1	High-Ratio Small Job	10	60	6.0
C2	Balanced Combo A	20	100	5.0
C3	Balanced Combo B	30	120	4.0



6.6 What-If Simulation

The user can change the available resource capacity and observe how the optimal project selection changes.

This demonstrates the practical impact of algorithmic optimization.

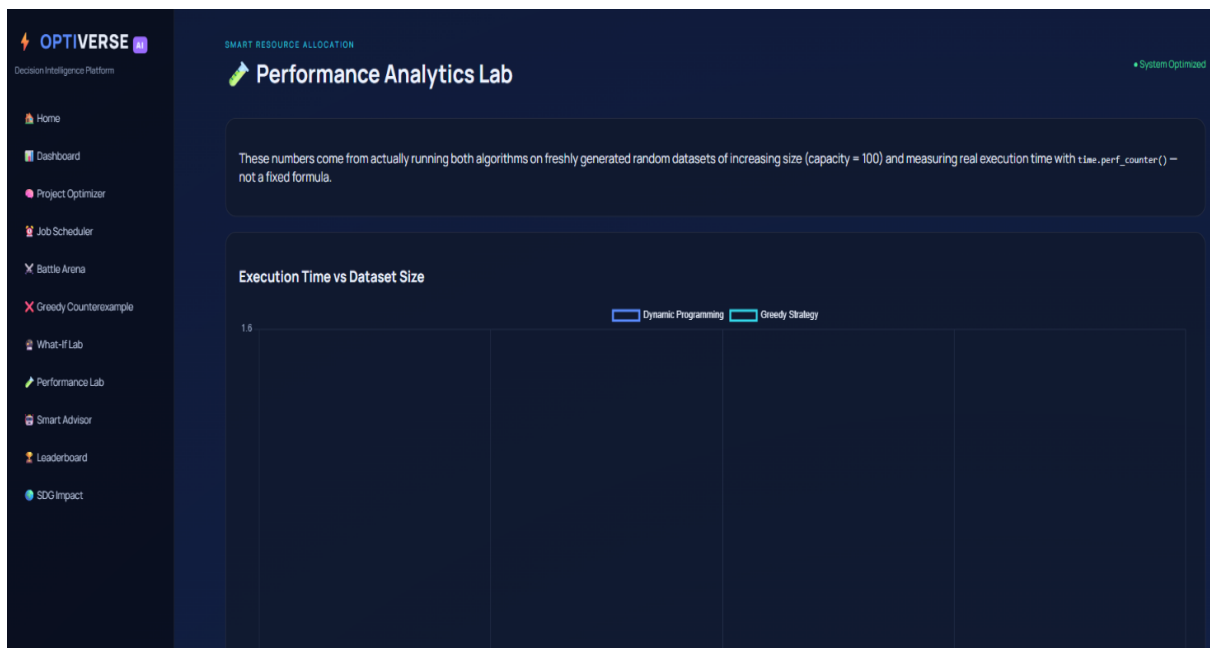


6.7 Performance Analytics

The system evaluates execution time for different dataset sizes.

Metrics include:

- Dataset size
- DP execution time
- Greedy execution time
- Scalability
- Resource utilization



Measured Results				
Dataset Size	DP Time (ms)	Greedy Time (ms)	DP Profit	Greedy Profit
10	0.3084	0.0184	502	502
20	0.6126	0.0206	792	792
30	0.9014	0.0297	1292	1260
40	1.4062	0.0431	1448	1377
50	1.5232	0.0458	1292	1292

Dynamic Programming

Solution quality: **Optimal**

Scalability: Time grows with $n \times \text{capacity}$ (pseudo-polynomial)

Space: $O(n \times C)$

Greedy Strategy

Solution quality: **Not guaranteed optimal** for 0/1 Knapsack (see Counterexample page)

Scalability: Faster, $O(n \log n)$

Space: $O(1)$ extra

OptiVerse AI • Smart Decision Allocation & Optimization System

6.8 Decision Advisor

The advisor converts algorithmic outputs into human-readable recommendations.

For example:

“Dynamic Programming is recommended when obtaining the optimal project combination is more important than minimizing computation time.”



Decision Intelligence Platform

- Home
- Dashboard
- Project Optimizer
- Job Scheduler
- Battle Arena
- Greedy Counterexample
- What-If Lab
- Performance Lab
- Smart Advisor
- Leaderboard
- SDG Impact

CS406/2 • DAA Project
DP • Greedy Strategies

SMART RESOURCE ALLOCATION

Smart Decision Advisor

Tell OptiVerse what matters most.

Is guaranteed maximum profit critical?

Yes — optimality is critical

How important is speed?

Medium

Resource capacity

Limited

Ask the Advisor

EXPLAINABLE RECOMMENDATION

Dynamic Programming - 0/1 Knapsack

Your scenario prioritizes guaranteed optimal profit under limited resources.

7. ALGORITHM AND PSEUDOCODE

7.1 0/1 KNAPSACK USING DYNAMIC PROGRAMMING

Algorithm

Let: $DP[i][c]$

represent the maximum profit obtainable using the first i projects with resource capacity c .

The recurrence is:

If: $resource_i \leq c$

then:

$$DP[i][c] = \max(DP[i-1][c], profit_i + DP[i-1][c - resource_i])$$

Otherwise:

$$DP[i][c] = DP[i-1][c]$$

Pseudocode:

Algorithm KNAPSACK(projects, capacity)

Input:

projects = list of projects

capacity = total available resource

Output:

maximum profit

selected projects

DP table

1. $n \leftarrow$ number of projects
2. Create DP table of size $(n + 1) \times (\text{capacity} + 1)$
3. Initialize all DP values to 0
4. For $i \leftarrow 1$ to n :
 $resource \leftarrow$ projects[$i-1$].resource
 $profit \leftarrow$ projects[$i-1$].profit
 For $c \leftarrow 0$ to capacity:
 If $resource \leq c$:
 $include \leftarrow profit + DP[i-1][c-resource]$
 $exclude \leftarrow DP[i-1][c]$
 $DP[i][c] \leftarrow \max(include, exclude)$
 Else:
 $DP[i][c] \leftarrow DP[i-1][c]$
5. $maximum_profit \leftarrow DP[n][\text{capacity}]$

6. Set $c \leftarrow \text{capacity}$
7. For $i \leftarrow n$ down to 1:
 - If $DP[i][c] \neq DP[i-1][c]$:
 - Select project i
 - $c \leftarrow c - \text{resource of project } i$
8. Reverse selected project list
9. Calculate total resources used
10. Calculate unused capacity
11. Return results and DP table

End Algorithm

7.2 JOB SEQUENCING WITH DEADLINES

Pseudocode

Algorithm JOB-SEQUENCING(jobs, slots)

Input:

jobs = list of jobs
 slots = available time slots

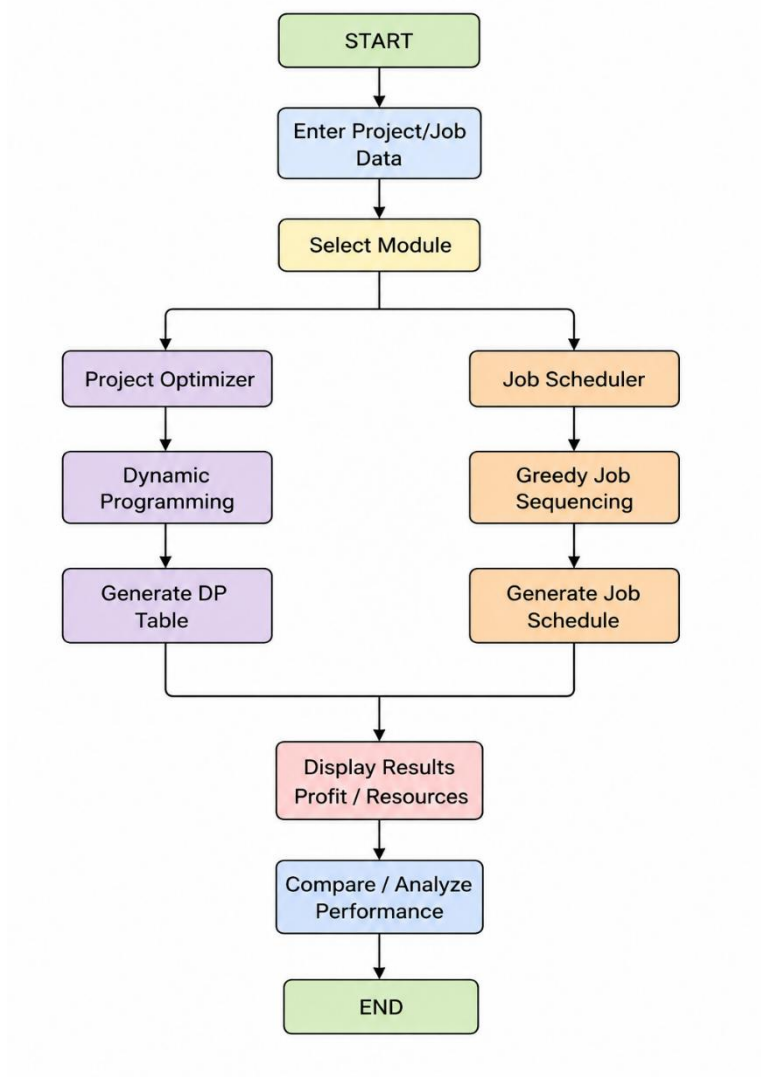
Output:

selected jobs
 job sequence
 total profit

1. Sort jobs in descending order of profit
2. Create schedule[1...slots]
 Initialize every slot as EMPTY
3. For each job in sorted jobs:
 - $\text{deadline} \leftarrow \text{minimum}(\text{job.deadline}, \text{slots})$
 - For position $\leftarrow \text{deadline}$ down to 1:
 - If schedule[position] is EMPTY:
 - schedule[position] $\leftarrow$ job
 - Break
4. selected_jobs $\leftarrow$ all non-empty schedule positions
5. total_profit $\leftarrow$ sum of profits of selected jobs
6. Return selected_jobs, schedule and total_profit

End Algorithm

7.3 FLOWCHART – MAIN SYSTEM



8. IMPLEMENTATION / SOURCE CODE AND ENVIRONMENT

8.1 Technology Stack

Component	Technology
Programming Language	Python
Backend	Flask
Frontend	HTML5, CSS3, JavaScript
Visualization	Chart.js / JavaScript
Data Storage	CSV
Algorithm Module	Python
Development Environment	Visual Studio Code
Browser	Google Chrome / Microsoft Edge
Version Control	Git / GitHub

8.2 Project Structure

OptiVerse_AI/

```
|
|
|— algorithms.py
|— app.py
|— requirements.txt
|— README.md
|
|— data/
|   |— projects.csv
|   |— projects_seed.csv
|   |— jobs.csv
|   └— jobs_seed.csv
|
|— static/
|   |— css/
|       └— style.css
```

```
|   └─ js/
|     └─ script.js
|
└─ templates/
    ├── base.html
    ├── index.html
    ├── dashboard.html
    ├── projects.html
    ├── jobs.html
    ├── battle.html
    ├── counterexample.html
    ├── simulator.html
    ├── performance.html
    ├── advisor.html
    ├── leaderboard.html
    └─ sdg.html
```

Implementation Note

The algorithmic logic is separated from the Flask application. This modular design makes the algorithms independently testable and allows the same optimization functions to be reused by different web modules.

9. TEST CASES AND EXPECTED / ACTUAL RESULTS

Test Case 1 – Normal Project Optimization

Input

Resource capacity: **50**

Project	Resource	Profit
P01 – AI Automation	18	95
P02 – Cloud Upgrade	12	70
P03 – Green Energy	20	100
P04 – Office Renovation	10	35
P05 – Cybersecurity	15	85

Expected Result

The system should select the combination with the highest total profit without exceeding resource capacity.

Actual Result

Selected projects:

- P01 – AI Automation
- P02 – Cloud Upgrade
- P03 – Green Energy

Resource used:

$$18 + 12 + 20 = 50$$

Maximum profit:

$$95 + 70 + 100 = \boxed{265}$$

Unused capacity:

$$50 - 50 = \boxed{0}$$

Result

Test Passed

Test Case 2 – Job Sequencing

Available slots: 3

Job	Deadline	Profit
J01 – Data Backup	2	40
J02 – Security Patch	1	70
J03 – Server Migration	2	55
J04 – Report Generation	3	30
J05 – System Audit	1	25

Actual Schedule

Slot	Job	Profit
1	J02 – Security Patch	70
2	J03 – Server Migration	55
3	J04 – Report Generation	30

Total profit:

$$70 + 55 + 30 = \boxed{155}$$

Result

Test Passed

Test Case 3 – Greedy vs Dynamic Programming

Capacity:

50

Dynamic Programming

- Profit = **265**
- Resource used = **50**
- Unused = **0**

Greedy

- Profit = **250**
- Resource used = **45**
- Unused = **5**

Metric	Dynamic Programming	Greedy
Profit	265	250
Resource Used	50	45
Unused Resource	0	5
Optimality	Guaranteed	Not guaranteed
Decision Method	Global optimization	Local ratio choice

Observation

Dynamic Programming produces a higher-quality solution because it evaluates combinations of projects rather than committing permanently to the locally best ratio.

10. GREEDY FAILURE COUNTEREXAMPLE

The following dataset is intentionally designed to show the limitation of Greedy for 0/1 Knapsack.

Capacity

$$C = 50$$

Project	Resource	Profit	Profit/Resource
C1	10	60	6.0
C2	20	100	5.0
C3	30	120	4.0

Greedy Result

Greedy chooses C1 first because its ratio is highest.

Then C2 is selected.

Total:

$$10 + 20 = 30$$

Profit:

$$60 + 100 = \boxed{160}$$

C3 cannot be added because:

$$30 + 30 > 50$$

Dynamic Programming Result

DP selects:

- C2
- C3

Resource: $20 + 30 = 50$

Profit: $100 + 120 = \boxed{220}$

Therefore: $220 > 160$

Conclusion

This experiment proves that the profit/resource ratio Greedy heuristic does **not guarantee an optimal solution for 0/1 Knapsack**.

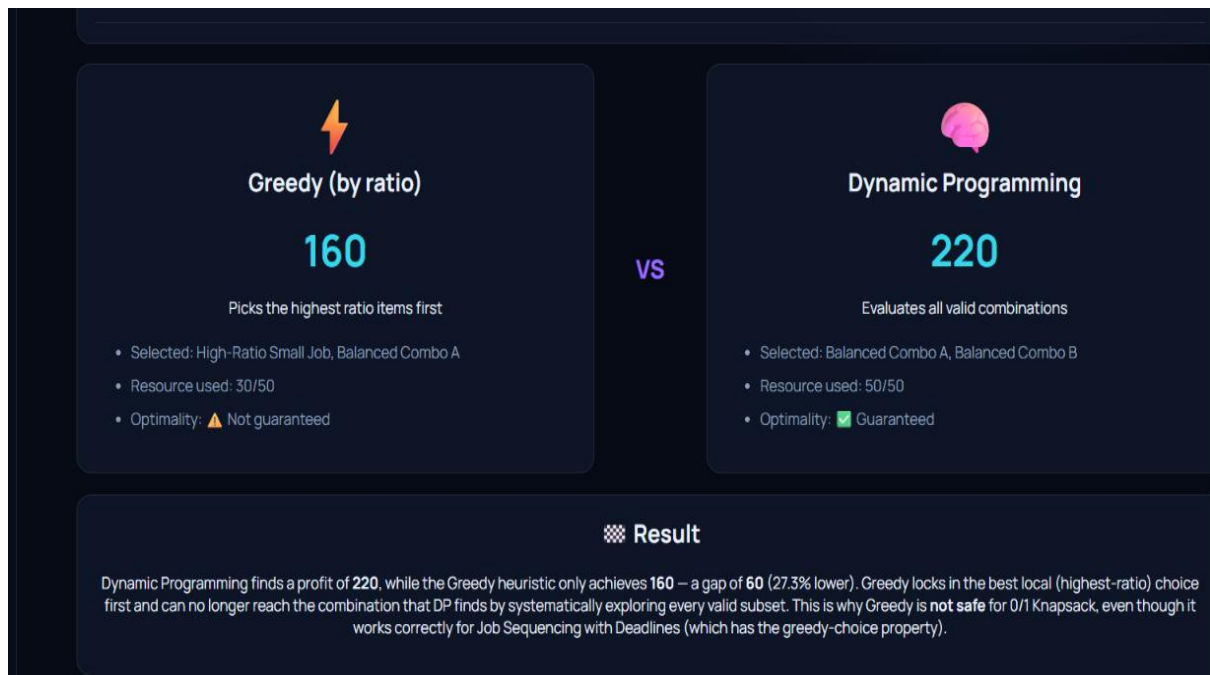


Figure 9: Greedy Failure Counterexample – DP = 220, Greedy = 160

11. COMPLETE DP TABLE

For your report, this is especially important because the assignment explicitly asks for the **complete DP table**.

Your application should show the complete matrix generated by the algorithm.

The table represents: $DP[i][c]$

where:

- i = number of projects considered
- c = available resource capacity
- Cell value = maximum profit possible

Interpretation

For example:

$DP[3][50] = 265$

means:

Using the first three projects and a resource capacity of 50, the maximum achievable profit is 265.

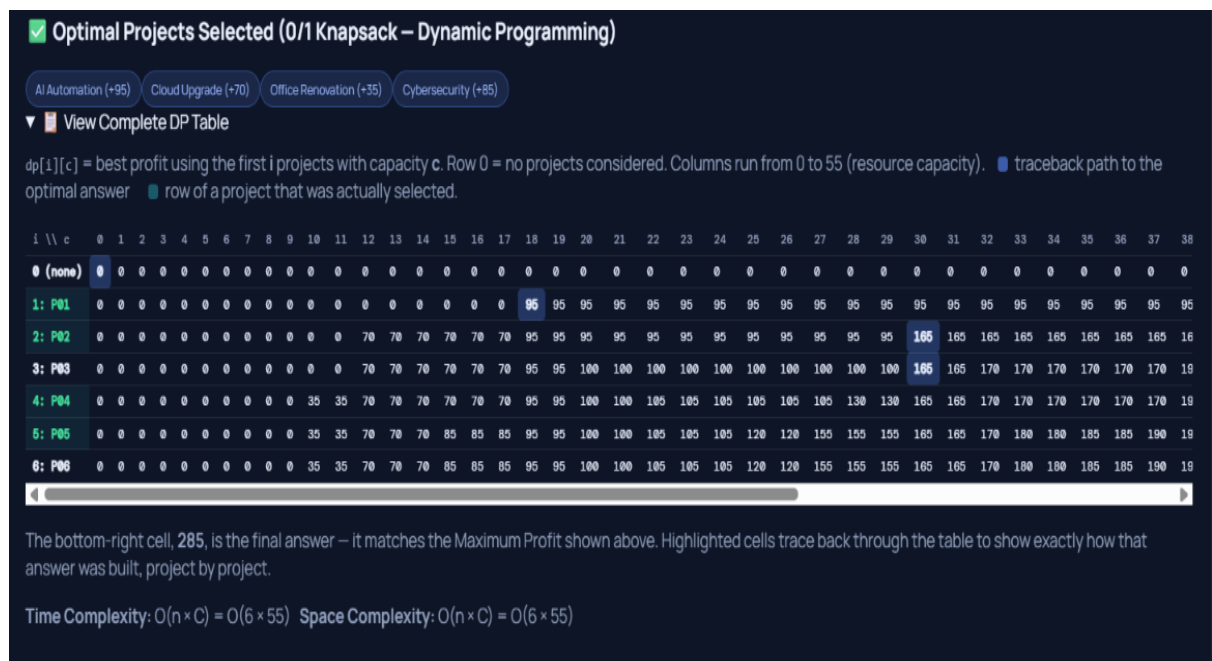


Figure 10: Complete 0/1 Knapsack Dynamic Programming Table

12. Analysis, Comparison, Trade-offs and Justification

12.1 Complexity Comparison

Algorithm	Time Complexity	Space Complexity
0/1 Knapsack DP	$O(nC)$	$O(nC)$
Job Sequencing Greedy	$O(n \log n + nd)$	$O(d)$

where:

- n = number of items/jobs
- C = resource capacity
- d = maximum deadline/available slots

12.2 Trade-off

Dynamic Programming

Advantages

- Guarantees optimal solution.
- Handles 0/1 selection correctly.
- Provides complete decision states.

Disadvantages

- Higher memory usage.
- Complexity depends on capacity.

Greedy

Advantages

- Faster.
- Simple implementation.
- Lower memory requirement.

Disadvantages

- No general optimality guarantee for 0/1 Knapsack.
- Local decisions can result in globally inferior solutions.

12.3 Final Justification

Dynamic Programming was selected for project optimization because the primary requirement is to obtain the maximum possible profit under a fixed resource capacity. Greedy was selected for job sequencing because deadline-based scheduling can be efficiently handled by prioritizing profitable jobs and assigning them to feasible slots. Therefore, the final system uses different algorithmic paradigms according to the characteristics of each optimization problem.

13. Broader Considerations / SDG Relevance

SDG 8 – Decent Work and Economic Growth

The system promotes efficient utilization of resources and prioritization of profitable activities, supporting improved productivity and operational efficiency.

SDG 9 – Industry, Innovation and Infrastructure

The system demonstrates how algorithmic optimization and digital decision-support tools can improve resource planning and infrastructure utilization.

14. Conclusion, Limitations and Possible Improvements

14.1 Conclusion

The OptiVerse AI system successfully demonstrates the application of Dynamic Programming and Greedy algorithms to practical optimization problems. The 0/1 Knapsack algorithm identifies the optimal combination of projects under a limited resource capacity, while the Greedy Job Sequencing algorithm generates a profitable schedule subject to deadlines.

Experimental comparison demonstrates that Dynamic Programming provides optimal project selection, whereas Greedy provides faster execution but does not guarantee optimality for 0/1 Knapsack. The integration of both algorithms into an interactive web application makes the algorithmic concepts easier to visualize, compare and evaluate.

14.2 Limitations

- DP memory increases with capacity.
- Current datasets are relatively small.
- Resource values are assumed to be integers.
- Execution time depends on hardware.

- Priority weights are predefined.
- Greedy is not universally optimal.

14.3 Possible Improvements

- Multi-resource Knapsack.
- Multi-objective optimization.
- Database integration.
- Machine-learning-based profit prediction.
- Adaptive priority weights.
- Larger datasets.
- Cloud deployment.
- Advanced scheduling algorithms.

Individual Reflection – J. Renuka Devi (192573007)

Design/Development Decisions:

I worked primarily on the Dynamic Programming component of the project. The 0/1 Knapsack formulation was selected because each project is either completely selected or rejected and the objective is to maximize profit within a limited resource capacity.

Challenges:

The main challenge was understanding the construction of the DP table and recovering the selected projects from the final solution.

Learning:

I gained practical understanding of optimal substructure, overlapping subproblems, recurrence relations, backtracking and complexity analysis.

SDG Connection:

Efficient resource allocation supports SDG 8 by improving productivity and SDG 9 through efficient use of infrastructure and resources.

CO Attainment:

The assignment directly contributed to CO4 by enabling me to apply Dynamic Programming to solve an optimization problem and evaluate its effectiveness experimentally.

B. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	10	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	15	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	20	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	15	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.
Efficiency, Testing & Validation	15	Performs appropriate complexity evaluation and systematic testing using suitable	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or

		data/input sizes; validates correctness, efficiency and scalability with clear evidence.			substantially incorrect.
Performance Improvement, Comparison & Final Decision	15	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	Good performance comparison and justification with minor limitations.	Basic comparison is provided, but improvement or final justification lacks depth.	Improvement is not demonstrated or the final algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results
3. Github Upload